

|                                                  |    |
|--------------------------------------------------|----|
| Full SoC Subsystem Integration.....              | 2  |
| 1. Title .....                                   | 2  |
| 2. Objective .....                               | 2  |
| 3. Theory (Multi-Level) .....                    | 2  |
| 3.1 Beginner — Simple Intuition.....             | 2  |
| 3.2 Intermediate — Engineering View.....         | 3  |
| 3.3 Advanced — Signal/Architecture Level .....   | 3  |
| 4. Architecture / Concept.....                   | 4  |
| Complete SoC Block Diagram.....                  | 4  |
| Memory Map (Definitive Reference) .....          | 5  |
| 5. Implementation.....                           | 5  |
| Files Used in This Module .....                  | 5  |
| Makefile Targets.....                            | 5  |
| 6. Lab / Hands-on Section .....                  | 6  |
| Step 1: Run Full SoC Simulation.....             | 6  |
| Expected Full Simulation Log .....               | 6  |
| Step 2: Run Structured Debug SoC.....            | 7  |
| Step 3: Open Waveform .....                      | 7  |
| Step 4: Run Synthesis.....                       | 7  |
| 7. Waveform / Verification Insight.....          | 8  |
| GTKWave Signal Groups for Full SoC Analysis..... | 8  |
| Key Timing Observations .....                    | 8  |
| 8. Common Issues & Debug Tips .....              | 9  |
| Synthesis Readiness Checklist.....               | 10 |
| 9. Summary .....                                 | 10 |

## Full SoC Subsystem Integration

**Course:** PicoRV32 SoC with AXI-Lite & UART Integration

### 1. Title

**Full SoC Subsystem Integration — End-to-End System Verification of PicoRV32 + AXI-Lite + UART**

### 2. Objective

By the end of this module, learners will be able to:

- Describe the complete PicoRV32 SoC hierarchy and all 9 RTL modules
- Explain how the address decoder routes CPU transactions to the correct slave
- Run `make soc` and `make debug_soc` to execute end-to-end system verification
- Interpret the complete simulation log showing greeting messages and PING echo
- Understand OpenLane synthesis flow and the ASIC-clean design requirements

### 3. Theory (Multi-Level)

#### 3.1 Beginner — Simple Intuition

At this point, every piece you've built in Modules 1–7 comes together into one system:

Firmware (C code written in Sub-module 7)

↓

PicoRV32 CPU (Sub-module 1) executes instructions

↓

CPU-to-AXI bridge (Sub-module 5) translates native bus to AXI-Lite

↓

AXI Interconnect routes to:

- ROM (stores the firmware instructions) [Sub-module 2 AXI slave]
- SRAM (scratch memory) [Sub-module 2 AXI slave]
- UART peripheral (Sub-module 3/6) [sends bytes to terminal]

The testbench (Sub-module 9) monitors UART output and verifies "PING" echo.

Think of it as assembling a complete computer from scratch and watching it run its first program.

#### 3.2 Intermediate — Engineering View

## Full RTL file list and roles:

| File                        | Lines | Role                                    |
|-----------------------------|-------|-----------------------------------------|
| rtl/picorv32.v              | ~3100 | PicoRV32 CPU (all RISC-V ISA logic)     |
| rtl/top.v                   | 333   | SoC top: CPU + bridge + wiring          |
| rtl/axi_lite_interconnect.v | 227   | 1-master, 3-slave AXI-Lite router       |
| rtl/axi_decoder.v           | 165   | One-hot address decoder (ROM/SRAM/UART) |
| rtl/rom.v                   | 212   | AXI-Lite ROM (firmware storage)         |
| rtl/sram.v                  | 145   | AXI-Lite SRAM (data + stack)            |
| rtl/uart_axi.v              | 250   | UART peripheral with AXI-Lite slave     |
| rtl/uart_tx.v               | 124   | UART 8N1 transmitter FSM                |
| rtl/uart_rx.v               | 143   | UART 8N1 receiver FSM with 2-FF sync    |

**ASIC-clean requirements (enforced throughout):** - No initial blocks (other than in TB or sim-only ifdef paths) - No inline register initializers (no `reg foo = 1'b0`) - All state driven by synchronous posedge `reset` - No combinational loops - All outputs have defined values in all states (no inferred latches)

### 3.3 Advanced — Signal/Architecture Level

**Complete address map (from `rtl/axi_decoder.v`):**

```
casez (addr)
  32'h0000_????: sel = 3'b001; // ROM (0x0000_0000 - 0x0000_FFFF)
  32'h0001_????: sel = 3'b010; // SRAM (0x0001_0000 - 0x0001_FFFF)
  32'h1000_000?: sel = 3'b100; // UART (0x1000_0000 - 0x1000_000F)
  default:      sel = 3'b001; // Default → ROM (safe)
endcase
```

**AXI interconnect routing logic (for write address channel):**

```
// One-hot fanout to all slaves simultaneously
assign s0_axi_awvalid = m_axi_awvalid & wr_sel[0]; // ROM
assign s1_axi_awvalid = m_axi_awvalid & wr_sel[1]; // SRAM
assign s2_axi_awvalid = m_axi_awvalid & wr_sel[2]; // UART
```

// Only the selected slave's awready propagates back to master

```
assign m_axi_awready = (wr_sel[0] & s0_axi_awready) |  
                        (wr_sel[1] & s1_axi_awready) |  
                        (wr_sel[2] & s2_axi_awready);
```

The **transaction tracking** registers (wr\_sel\_r, rd\_sel\_r, wr\_active, rd\_active) ensure that even if the address changes after a transaction starts, the response is correctly routed back to the master.

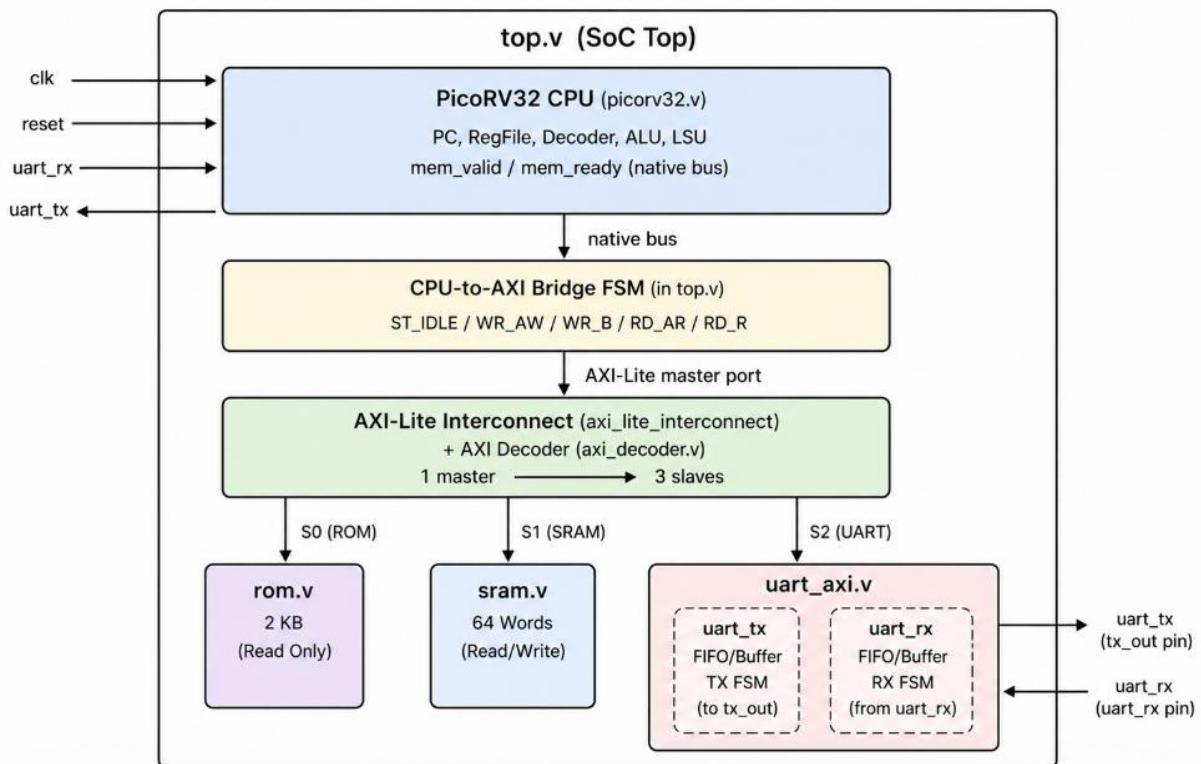
**Reset sequence in rtl/top.v:** - reset input is active-HIGH - Drives resetn = ~reset to PicoRV32 (which uses active-LOW reset) - All slaves see active-HIGH reset

**PicoRV32 SoC port pinout (rtl/top.v):**

```
module top (  
    input  clk,           // 50 MHz system clock  
    input  reset,         // Active-HIGH synchronous reset (drive for ≥ 2 cycles)  
    output uart_tx,       // UART serial output (to terminal or testbench monitor)  
    input  uart_rx        // UART serial input (from testbench or host)  
);
```

## 4. Architecture / Concept

### Complete SoC Block Diagram



## Memory Map (Definitive Reference)

| Range                           | Slave | Size                             | Interface   |
|---------------------------------|-------|----------------------------------|-------------|
| 0x0000_0000<br>–<br>0x0000_FFFF | ROM   | 64KB (logical) / 2KB<br>(depth)  | AXI-Lite S0 |
| 0x0001_0000<br>–<br>0x0001_FFFF | SRAM  | 64KB (logical) / 256B<br>(depth) | AXI-Lite S1 |
| 0x1000_0000<br>–<br>0x1000_000F | UART  | 16 bytes                         | AXI-Lite S2 |

**Note:** ROM depth is MEM\_DEPTH=2048 words = 8KB but address window is 64KB (upper bits ignored). SRAM depth is 64 words = 256 bytes.

## 5. Implementation

### Files Used in This Module

| File                        | Role                                                          |
|-----------------------------|---------------------------------------------------------------|
| rtl/top.v                   | SoC top-level (CPU + bridge + interconnect wiring)            |
| rtl/axi_lite_interconnect.v | AXI-Lite routing fabric                                       |
| rtl/axi_decoder.v           | One-hot address decode                                        |
| rtl/rom.v                   | AXI ROM (dual-path: sim uses \$readmemh, synth uses case-ROM) |
| rtl/sram.v                  | AXI SRAM with byte-enable write                               |
| rtl/uart_axi.v              | UART with AXI-Lite slave interface                            |
| rtl/uart_tx.v               | TX FSM                                                        |
| rtl/uart_rx.v               | RX FSM with 2-FF sync                                         |
| rtl/picorv32.v              | Full PicoRV32 core                                            |
| tb/tb_top.v                 | Full system testbench with UART byte monitor + PING test      |
| tb/tb_debug.v               | Structured debug testbench with more verbose signal logging   |
| rtl/rom.hex                 | Firmware binary in hex format (generated by make fw)          |

### Makefile Targets

```
make soc           # fw + verilate all RTL + tb_top.v → sim_soc → tb_top.vcd
make debug_soc    # fw + verilate all RTL + tb_debug.v → sim_verilator →
tb_top_debug.vcd
make synth        # Yosys synthesis using openlane_design/src/ with -D
SYNTHESIS
```

## 6. Lab / Hands-on Section

### Step 1: Run Full SoC Simulation

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc  
make soc
```

This command: 1. `cd fw && make` — builds `rom.hex` 2. `verilator --binary --trace --timing` — compiles ALL 9 RTL files + `tb_top.v` 3. Runs `./obj_soc/sim_soc +romhex=rtl/rom.hex` → produces `tb_top.vcd` 4. Converts to FST: `vcd2fst tb_top.vcd tb_top.fst` 5. Opens GTKWave: `gtkwave tb_top.fst` &

### Expected Full Simulation Log

```
-----  
UART SoC Simulation Started  
-----  
[ROM] loaded from plusarg: rtl/rom.hex  
  
[TB] UART Monitor Started  
-----  
[TB] RX byte: H  
[TB] RX byte: e  
[TB] RX byte: l  
[TB] RX byte: l  
[TB] RX byte: o  
[TB] RX byte:  
[TB] RX byte: D  
[TB] RX byte: e  
[TB] RX byte: e  
[TB] RX byte: p  
[TB] RX byte: a  
[TB] RX byte: k  
[TB] RX byte:  
[TB] RX byte: f  
[TB] RX byte: r  
[TB] RX byte: o  
[TB] RX byte: m  
[TB] RX byte:  
[TB] RX byte: N  
[TB] RX byte: i  
[TB] RX byte: e  
[TB] RX byte: l  
[TB] RX byte: i  
[TB] RX byte: t  
[TB] RX byte: !  
Hello Deepak from Nielit!
```

```
[TB] Greeting message count = 1
```

...

[TB] Greeting message count = 10

```
=====
10 GREETING MESSAGES RECEIVED
UART OUTPUT VERIFIED
=====
```

```
-----
[TB] Greeting phase finished
-----
```

```
-----
[TB] FIFO flushed - sending command : PING
-----
```

```
[TB] RX byte: P
[TB] RX byte: I
[TB] RX byte: N
[TB] RX byte: G
```

```
=====
TEST PASSED
Echo received : PING
=====
```

- Verilator: \$finish at ~23ms simulated time; walltime ~1.2s

## Step 2: Run Structured Debug SoC

`make debug_soc`

This uses `tb/tb_debug.v` which provides more verbose signal-level logging during the simulation, producing `tb_top_debug.fst`.

## Step 3: Open Waveform

`make wave` *# Opens tb\_top.fst in GTKWave*

For the debug waveform:

`gtkwave tb_top_debug.fst &`

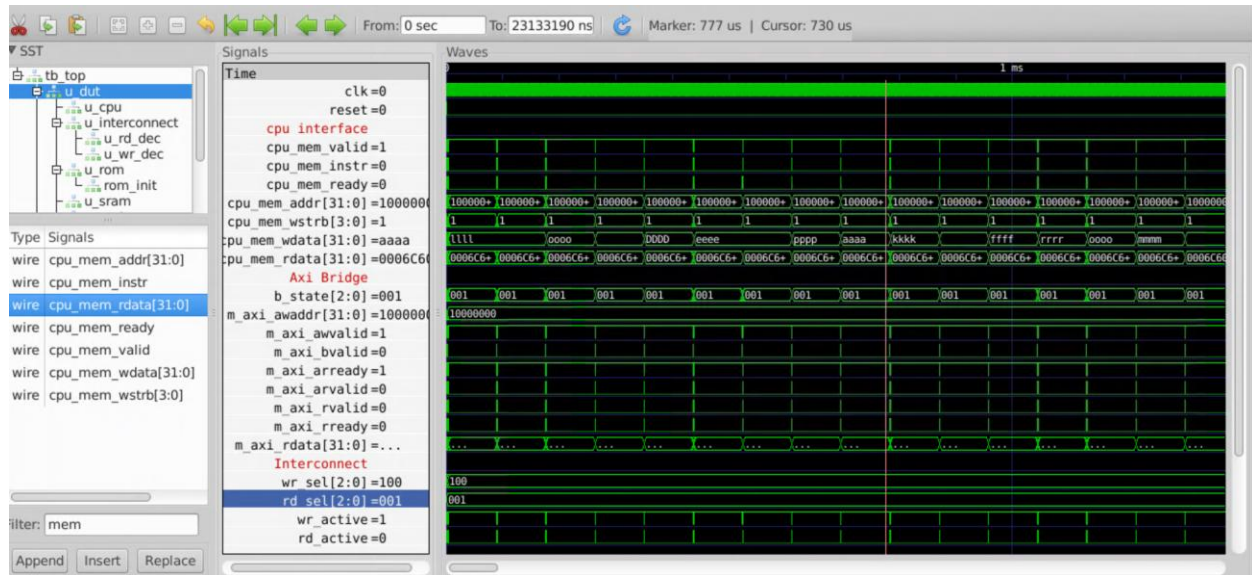
## Step 4: Run Synthesis

`make synth`

This runs Yosys synthesis using: - Source files in `openlane_design/src/` - `-D SYNTHESIS` flag (activates case-ROM in `rom.v`, bypasses `$readmemh`) - Outputs: `top.json` (netlist) and `synth.v` (synthesized Verilog)



## 7. Waveform / Verification Insight



### GTKWave Signal Groups for Full SoC Analysis

#### Group 1: CPU Interface

```
clk
reset
u_dut.u_cpu.mem_valid
u_dut.u_cpu.mem_instr
u_dut.u_cpu.mem_ready
u_dut.u_cpu.mem_addr[31:0]
u_dut.u_cpu.mem_wstrb[3:0]
u_dut.u_cpu.mem_wdata[31:0]
u_dut.u_cpu.mem_rdata[31:0]
```

#### Group 2: AXI Bridge

```
u_dut.b_state[2:0]      ← 0=IDLE, 1=WR_AW, 2=WR_B, 3=RD_AR, 4=RD_R
u_dut.m_axi_awaddr[31:0]
u_dut.m_axi_awvalid
u_dut.m_axi_awready
u_dut.m_axi_bvalid
u_dut.m_axi_arvalid
u_dut.m_axi_arready
u_dut.m_axi_rvalid
u_dut.m_axi_rready
u_dut.m_axi_rdata[31:0]
```

#### Group 3: Interconnect

```
u_dut.u_interconnect.wr_sel[2:0]  ← one-hot: 001=ROM, 010=SRAM, 100=UART
u_dut.u_interconnect.rd_sel[2:0]
```



```
u_dut.u_interconnect.wr_active
u_dut.u_interconnect.rd_active
```

#### Group 4: UART

```
uart_tx_pin      ← top-level serial output
u_dut.u_uart.buf_valid
u_dut.u_uart.rx_buf_valid
u_dut.u_uart.u_tx.state[1:0]
u_dut.u_uart.u_rx.state[1:0]
```

#### Key Timing Observations

| Event                   | Approximate Time | What to See                               |
|-------------------------|------------------|-------------------------------------------|
| Reset released          | ~200 ns          | b_state goes to IDLE;<br>mem_addr = 0x0   |
| First instruction fetch | ~220 ns          | m_axi_araddr=0x0000_0000, rd_sel=001      |
| First UART write ('H')  | ~few $\mu$ s     | m_axi_awaddr=0x1000_0000, wr_sel=100      |
| UART TX serial bit      | ~first byte      | uart_tx_pin drops to 0 (start bit)        |
| All 10 greetings sent   | ~23 ms sim       | 260 bytes $\times$ 86.8 $\mu$ s = 22.6 ms |
| PING echo completes     | ~23.4 ms         | TEST PASSED log                           |

#### 8. Common Issues & Debug Tips

| Issue                             | Likely Cause        | Fix                                                 |
|-----------------------------------|---------------------|-----------------------------------------------------|
| make soc fails:<br>ROM not loaded | +romhex= path wrong | Check Makefile line 202:<br>+romhex=\$(RTL)/rom.hex |
| Simulation produces 0 greetings   | Firmware not built  | Run make fw first; verify rtl/rom.hex is non-empty  |
| CPU stuck after reset             | Bridge FSM bug      | Add \$display for b_state transitions               |

| Issue                                     | Likely Cause                           | Fix                                                    |
|-------------------------------------------|----------------------------------------|--------------------------------------------------------|
| wr_sel shows 3'b000                       | Address not decoded                    | Check axi_decoder.v — does address match any case?     |
| UART TX always 1 (idle)                   | buf_valid never set                    | Trace aw_got and w_got in uart_axi.v                   |
| bvalid never seen                         | Wrong wr_sel causing wrong slave       | Verify interconnect routes to UART for 0x10000000      |
| Synthesis: “uart_tx has no driver”        | Port name collision (known, fixed)     | Already resolved: tx_out rename in uart_axi.v          |
| OpenLane ABC error: combinational network | Timing retime command on mapped FFs    | Non-fatal; documented in uart_tx_no_driver_analysis.md |
| Verilator simulation is slow              | Large VCD from tb_mem_soc.vcd (142 MB) | Use FST format: vcd2fst then view .fst file            |

### Synthesis Readiness Checklist

- ☐ No inline reg initializers (= value) in any RTL
- ☐ No initial blocks in RTL (only in testbenches)
- ☐ All outputs have defined values in all states (no latches)
- ☐ Simulation passes: 10 greetings + PING echo → TEST PASSED
- ☐ synthesis path in rom.v tested: `make synth` runs without errors
- ☐ No multi-driven nets (Verilator -Wall finds these)
- ☐ Reset is synchronously sensed with correct polarity

## 9. Summary

| Module           | Component                   | Status                                      |
|------------------|-----------------------------|---------------------------------------------|
| PicoRV32 CPU     | rtl/picorv32.v              | Parameterized RV32I; boots from 0x0000_0000 |
| CPU-AXI Bridge   | rtl/top.v (lines 91–224)    | 5-state FSM; write + read paths             |
| AXI Interconnect | rtl/axi_lite_interconnect.v | 1M → 3S; address-tracked routing            |
| Address Decoder  | rtl/axi_decoder.v           | Combinational one-hot; casez match          |
| ROM              | rtl/rom.v                   | Dual-path: sim=\$readmemh, synth=case-ROM   |

| Module            | Component                                 | Status                                                         |
|-------------------|-------------------------------------------|----------------------------------------------------------------|
| SRAM              | rtl/sram.v                                | 64-word× 32-bit;<br>byte-enable write                          |
| UART peripheral   | rtl/uart_axi.v +<br>uart_tx.v + uart_rx.v | AXI slave; 1-byte TX<br>buffer; 2-FF RX sync                   |
| Firmware          | fw/main.c + crt0.S +<br>link.ld           | 167 bytes; greets<br>10× then echoes                           |
| Testbench         | tb/tb_top.v                               | UART monitor +<br>PING test → TEST<br>PASSED                   |
| <b>Lab Result</b> | Full system                               | 10 greetings verified<br>+ PING echoed →<br><b>TEST PASSED</b> |

This is the culmination of the course. The full design is ASIC-clean, simulation-verified, and synthesis-ready. The same RTL flows directly into OpenLane2 for physical design.

Previous: [Sub-module 7 — Firmware](#)

Next: [Sub-module 9 — Waveform Analysis](#)